Generics, Lambda Expressions

Chapter 2

SE411, V3

Fall 2026-27

Outline

- Generics
- Lambda Expressions

Learning Outcomes

- CLO1: Define and understand the fundamentals of software construction concepts
- CLO4: Evaluate the key construction concepts in terms of best practices
- CLO5: Analyze and improve existing or new software projects
- using various software research techniques to ensure quality
- and innovation

Lambda expressions *(parameter1, parameter2) -> { code block }*

- Lambda Expressions were added in Java 8.
- A lambda expression is ***like*** a function, it takes in parameters and executes some code.
- It doesn't need a name and can be implemented where needed.

```
ArrayList<Integer> numbers = new ArrayList<Integer>();
numbers.add(5);
numbers.add(9);
numbers.add(8);
numbers.forEach( (n) -> { System.out.println(n); } );
```

!!! Lambda expressions can be stored in variables if the variable's type is an interface which has only one abstract method. The lambda expression should have the same number of parameters and the same return type as that method. Calling the interface's method will run the lambda expression.

Lambda Expression

```
interface Calculator {
    int calculate(int a, int b);
}

Calculator add = (a, b) -> a + b;

System.out.println(add.calculate(5, 3));
```

!!! Lambda expressions can be stored in variables if the variable's type is an interface which has only one abstract method. The lambda expression should have the same number of parameters and the same return type as that method. Calling the interface's method will run the lambda expression.

Why it works? (UPDATE EXAMPLE)

```
// parent class of Button
public abstract class ButtonBase extends Labeled {

    public final void setOnAction(EventHandler<ActionEvent> value) {
        onActionProperty().set(value);
    }
…
```

```
button.setOnAction(e -> {
        System.out.println(button.getText())
});
```

```
@FunctionalInterface
public interface EventHandler<T extends Event> extends EventListener {

    void handle(T event);

}
```

!!! Lambda expressions can be stored in variables if the variable's type is an interface which has only one abstract method. The lambda expression should have the same number of parameters and the same return type as that method. Calling the interface's method will run the lambda expression.

Equivalent code to Lambda Expressions with Java 7 or less

```
// In Java 8 and above:
btnAlert.setOnAction(event -> new AlertBox().display("Warning", "This is an alert!"));
```

THIS is Equivalent to:

```
// In Java 7 and below:
btnAlert.setOnAction(new EventHandler<ActionEvent>() {
        @Override
        public void handle(ActionEvent event) {
            new AlertBox().display("Warning", "This is an alert!");
        }
});
```

Exercise 1

```
interface Greeting {
    void sayHello(String name);
}

public class Main {
    public static void main(String[] args) {

        // TODO: Create a lambda expression
        Greeting greeting = _____;

        greeting.sayHello("Asma");
    }
}
```

Solution Exercise 1

```
interface Greeting {
    void sayHello(String name);
}

public class Main {
    public static void main(String[] args) {

        // TODO: Create a lambda expression
        Greeting greeting = name -> System.out.println("Hello " + name);
        greeting.sayHello("Asma");
    }
}
```

Exercise 2

```
interface NumberChecker {
    boolean check(int number);
}

public class Main {
    public static void main(String[] args) {

        // TODO: Create a lambda that checks if a number is even
        NumberChecker even = _____;

        System.out.println(even.check(10));
        System.out.println(even.check(7));
    }
}
```

Solution Exercise 2

```
interface NumberChecker {
    boolean check(int number);
}

public class Main {
    public static void main(String[] args) {

        // TODO: Create a lambda that checks if a number is even
        NumberChecker even = number -> number % 2 == 0;

        System.out.println(even.check(10));
        System.out.println(even.check(7));
    }
}
```

Generics

- Generics is a technique for expressing abstraction where *the code is invariant with respect to the specific **type** of certain entities*.
- **Some important things to remember about generics:**
  - *Generic type parameters do not specify a particular entity type but rather, are placeholders for any type*, perhaps within some boundaries.
  - *Generic type parameters enforce consistent type requirements across the affected code*.
  - *Generic type parameters are set to specific concrete types at construction time.* When an actual concrete object is instantiated from generic code, a particular value for the generic type parameter(s) **must** be supplied.

Example

```
class Box<T> {
    T value;

    void set(T value) {
        this.value = value;
    }

    T get() {
        return value;
    }
}
Box<Integer> intbox = new Box<>();
Box<String> stringbox = new Box<>();
```

```
intbox.set(10);       // correct
intbox.set("Hello");  // error
```

Before generics

- Before generic classes were added to Java, generic programming was achieved with *inheritance*. The ArrayList class simply maintained an array of Object references:

```
public class ArrayList // before generic classes
{
        private Object[] elementData;
        . . .
        public Object get(int i) { . . . }
        public void add(Object o) { . . . }
}
```

- This approach has two problems. A cast is necessary whenever you retrieve a value:

```
ArrayList files = new ArrayList();
String filename = (String) files.get(0);
```

- Moreover, there is no error checking. You can add values of any class:

```
files.add(new File(". . ."));  // an error will only appear when
                               // trying to cast this object back to String
```

Example: generics-02

Generic Collections

The main motivation for adding generic types to Java was the **Java Collections Framework**. This is a built-in library of different abstractions and implementations for **collections**.

Generics were introduced in Java 5 to make the Collections Framework more effective. Prior to this time, the interface Collection looked like the following:

```
/**  Collection before Java 5. */
interface Collection {

   /** Add object o to the collection. Return true if this changes
    * the state of the collection. */
   boolean add(Object o);

   /** Remove object o from the collection. Return true if this
changes
    * the state of the collection. */
   boolean remove(Object o);

  ...
}
```

```
Collection c = new ArrayList();
c.add("Riyadh");
c.add(100);
c.add(new Button());
```

```
/** Collection with generics */
public interface Collection<T> {


   public void add(T element);

   public boolean remove(T element);

   ...
}
```

```
Collection<String> names = new ArrayList<>();
names.add("Ali"); // correct
names.add("Sara"); // correct
names.add(100); // error
```

Abstractions using generics

Generics also help us when we are implementing abstractions.
Let's consider implementing the Collection interface using a **linked list**.
First, we will want generic linked list nodes that can contain data of an arbitrary type:

```
class Node<T> {

   T data;
   Node<T> next; // may be null

   /** Creates a linked list node containing
data d,
    *  where n is the next node.
   */
     Node(T d, Node<T> n) {
           data = d; next = n;
     }
}
```

```
class LList<T> implements Collection<T> {

   int size = 0;
   Node<T> head = null; // first Node; null if list is empty

     boolean add(T x) {
        head = new Node<T>(x, head);
        size++;
     }
…
}
```

- Then, we can use the linked-list with any type:

```
LList<Product> listOfProducts = new Llist<Product>();

Product p1 = new Product();
listOfProducts.add(p1);
```

Source Code: Generics_01->pair

Creating a generic class

- Let's create a Pair class that holds two attributes:

```
public class Pair<T>
{
	private T first;
	private T second;

	public Pair() { first = null; second = null; }
	public Pair(T first, T second) {
		this.first = first;
		this.second = second;
	}
	public T getFirst() { return first; }
	public T getSecond() { return second; }
	public void setFirst(T newValue) { first = newValue; }
	public void setSecond(T newValue) { second = newValue; }
}
```

- To instantiate this class we have to specify the substituting type:

Pair<String> or **Pair<Button>**

Exercise 3:

- What if we want to be able to also support different types for attributes first and second. How can we redesign the class Pair?

- The class starts by:

```
public class Pair2<T, S> { …..}
```

Source Code: Generics_01->pair2

Solution exercise 3:

```
public class Pair2<T, S> {

    private T first;
    private S second;

    public Pair2() {};
    public Pair2(T first, S second) {
        this.first = first;
        this.second = second;
    }
    public T getFirst() { return first; }
    public S getSecond() { return second; }
    public void setFirst(T newValue) { this.first = newValue; }
    public void setSecond(S newValue) { this.second = newValue; }
}
```

- Now, we can create pairs of (button, action) for example:

Pair2<Button, Action>

Generic Methods

- We can also define a single method with type parameters

```
class ArrayAlg {
        public static <T> int count(T... a)
        {
                return a.length;
        }

        public static <T> T getFirst(T... a)
        {
                return a[0];
        }

}
```

The `T...` a syntax allows the method to accept a variable number of arguments of type T.

<T> is placed before the return type

```
public class Main {
    public static void main(String[] args) {

        int n = ArrayAlg.count(10, 20, 30);
        System.out.println(n);

        String first = ArrayAlg.getFirst("Ali", "Sara", "Omar");
        System.out.println(first);
    }
}
```

Generics subclassing

- Consider the following method:

```
private void printList(List<Object> myList) {
    System.out.println("Something….");
}
```

- You may try to do the following:

```
List<Integer> intList = new ArrayList<>();
intList.add(3);
printList(intList);      // An error will be shown here
```

- The reason is the following: Even if Integer is a subclass of Object, **List<Integer>** is not a subclass of **List<Object>**, this is where we need to use **wildcards**.

Generics Wildcards

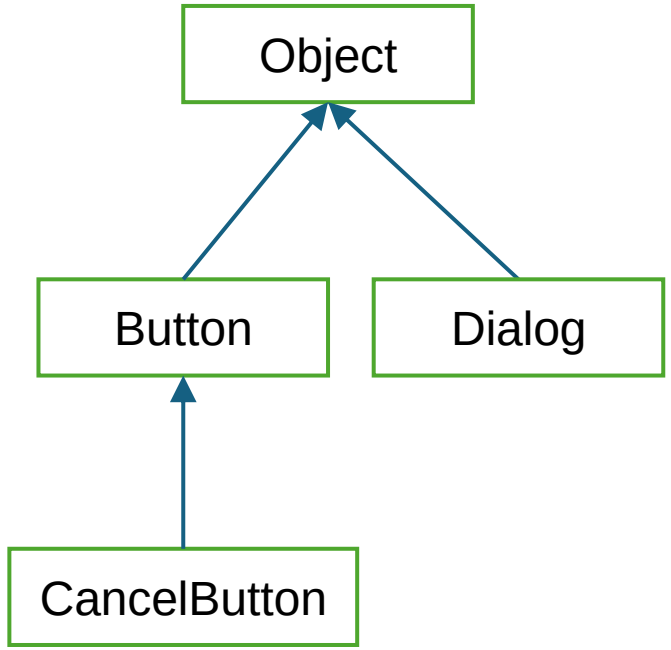
- Consider the following method:

```
Private void printList(List<?> myList) {
    System.out.println("Something….");
}
```

- The **?** (wildcard) can be replaced by any type.
- You may try to do the following:

```
List<Integer> intList = new ArrayList<>();
intList.add(3);
printList(intList);      // Now we can do this
```

Bound Wildcards



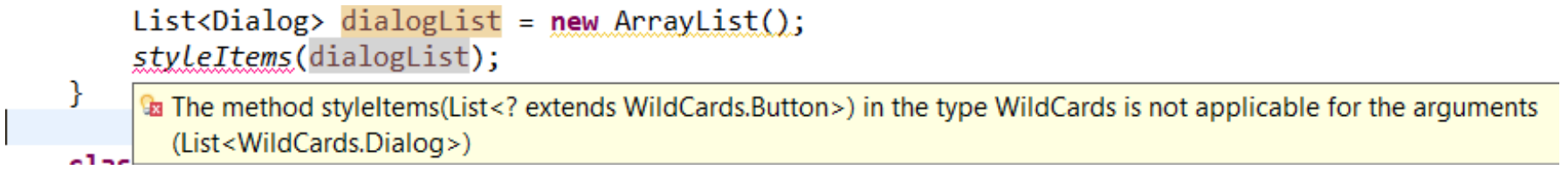
- Consider the following method (*anything that extends Button*):

```
private void styleListItems(List<? extends Button> myList) {
    // apply some style on all the elements of the list
}
```

- You may try to do the following:

```
List<CancelButton> cancelList = new ArrayList<>();
styleListItems(cancelList);      // Now we can do this

List<Dialog> dialogList = new ArrayList<>();
styleListItems(dialogList);      // But this is illegal as Dialog is not a subclass of Button
```



AI & Functional Java: Powerful, but Verify

- Generics and lambdas are verbose and pattern-heavy, where AI is very productive.
- **AI is good at:**
  - Converting Java 5 legacy code into modern Java using new concepts such as lambdas.
  - Generic-ifying a class or method.
- **But evaluate every rewrite:**
  - Does it preserve semantics for null and empty inputs?
  - Is the stream chain readable, or correct but too condensed for human readability?
- **Not every AI one-liner is an improvement, prefer clarity.**

Generic collections

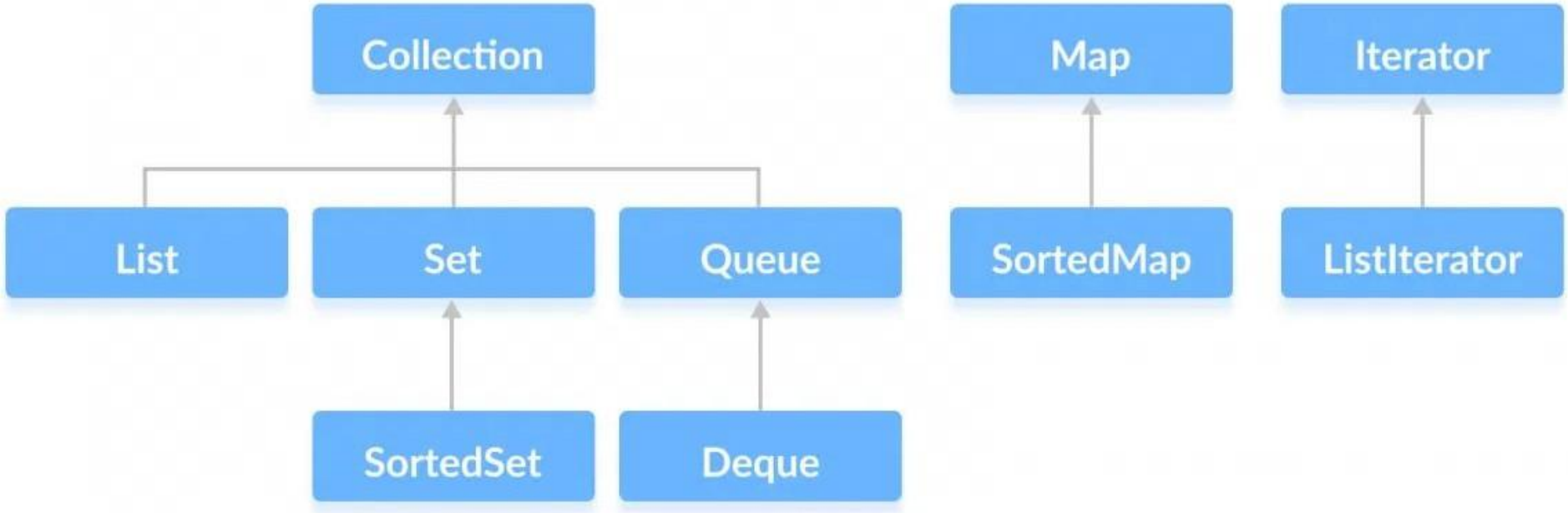
Java Collection Framework

- The Java Collection Framework (JCF) provides a unified architecture for managing and manipulating groups of objects
- JCF provides a set of Interfaces and Classes to implement various data structures and algorithms.
- These interfaces include several methods to perform different operations on collections.
- For example, the LinkedList class of the collections framework provides the implementation of the doubly-linked list data structure.

Interface	Description
`Collection`	The root interface in the collections hierarchy from which interfaces `Set`, `Queue` and `List` are derived.
`Set`	A collection that does *not* contain duplicates.
`List`	An ordered collection that *can* contain duplicate elements.
`Map`	A collection that associates keys to values and *cannot* contain duplicate keys. `Map` does not derive from `Collection`.
`Queue`	Typically a *first-in, first-out* collection that models a *waiting line*; other orders can be specified.

Fig. 16.1 | Some collections-framework interfaces.

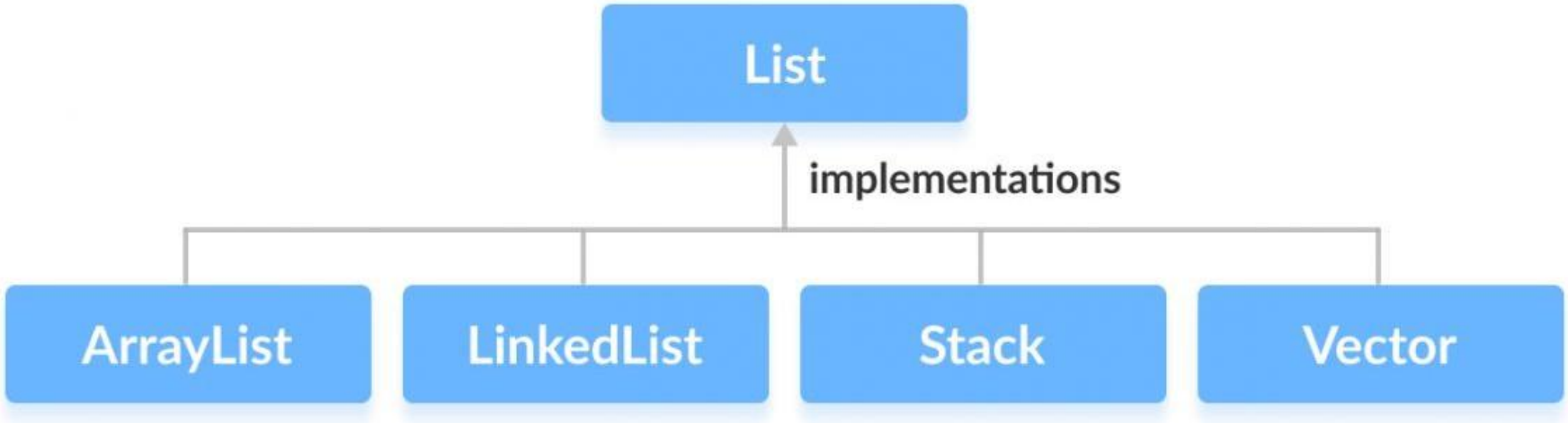
Java Collection Framework



Java Iterator Interface

- The Iterator interface provides methods that can be used to access elements of collections.

Classes that Implement List



Example: Using List Interface

- We must import java.util.List package in order to use List

```
//ArrayList implementation of List
List<String> list1 = new ArrayList<>();

//LinkedList implementation of List
List<String> list2 = new LinkedList<>();
```

- Here, we have created objects list1 and list2 of classes ArrayList and LinkedList. These objects can use the functionalities of the List interface.

Example: Using List Interface

Methods	Description
add()	adds an element to a list
addAll()	adds all elements of one list to another
get()	helps to randomly access elements from lists
iterator()	returns iterator object that can be used to sequentially access elements of lists
set()	changes elements of lists
remove()	removes an element from the list
removeAll()	removes all the elements from the list
clear()	removes all the elements from the list (more efficient than removeAll())
size()	returns the length of lists

Example: Using List Interface

```java
import java.util.List;
import java.util.ArrayList;

class Main {

    public static void main(String[] args) {
        // Creating list using the ArrayList class
        List<Integer> numbers = new ArrayList<>();

        // Add elements to the list
        numbers.add(1);
        numbers.add(2);
        numbers.add(3);
        System.out.println("List: " + numbers);

        // Access element from the list
        int number = numbers.get(2);
        System.out.println("Accessed Element: " + number);

        // Remove element from the list
        int removedNumber = numbers.remove(1);
        System.out.println("Removed Element: " + removedNumber);
    }
}
```

Example: Using List Interface

```java
import java.util.List;
import java.util.LinkedList;

class Main {

    public static void main(String[] args) {
        // Creating list using the LinkedList class
        List<Integer> numbers = new LinkedList<>();

        // Add elements to the list
        numbers.add(1);
        numbers.add(2);
        numbers.add(3);
        System.out.println("List: " + numbers);

        // Access element from the list
        int number = numbers.get(2);
        System.out.println("Accessed Element: " + number);

        // Using the indexOf() method
        int index = numbers.indexOf(2);
        System.out.println("Position of 3 is " + index);

        // Remove element from the list
        int removedNumber = numbers.remove(1);
        System.out.println("Removed Element: " + removedNumber);
    }
}
```



Good Programming Practice

Avoid reinventing the wheel—rather than building your own data structures, use the interfaces and collections from the Java collections framework, which have been carefully tested and tuned to meet most application requirements.

```
1  // Fig. 16.2: CollectionTest.java
2  // Collection interface demonstrated via an ArrayList object.
3  import java.util.List;
4  import java.util.ArrayList;
5  import java.util.Collection;
6  import java.util.Iterator;
7
8  public class CollectionTest {
9     public static void main(String[] args) {
10       // add elements in colors array to list
11       String[] colors = {"MAGENTA", "RED", "WHITE", "BLUE", "CYAN"};
12       List<String> list = new ArrayList<String>();
13
14       for (String color : colors) {
15          list.add(color); // adds color to end of list
16       }
17
```

Fig. 16.2 | `Collection` interface demonstrated via an `ArrayList` object. (Part 1 of 4.)

```
18          // add elements in removeColors array to removeList
19          String[] removeColors = {"RED", "WHITE", "BLUE"};
20          List<String> removeList = new ArrayList<String>();
21
22          for (String color : removeColors) {
23             removeList.add(color);
24          }
25
26          // output list contents
27          System.out.println("ArrayList: ");
28
29          for (int count = 0; count < list.size(); count++) {
30             System.out.printf("%s ", list.get(count));
31          }
```

Fig. 16.2 | `Collection` interface demonstrated via an `ArrayList` object. (Part 2 of 4.)

```
32
33          // remove from list the colors contained in removeList
34          removeColors(list, removeList);
35
36          // output list contents
37          System.out.printf("%n%nArrayList after calling removeColors:%n");
38
39          for (String color : list) {
40             System.out.printf("%s ", color);
41          }
42       }
43
```

Fig. 16.2 | `Collection` interface demonstrated via an `ArrayList` object. (Part 3 of 4.)

```
44      // remove colors specified in collection2 from collection1
45      private static void removeColors(Collection<String> collection1,
46         Collection<String> collection2) {
47         // get iterator
48         Iterator<String> iterator = collection1.iterator();
49
50         // loop while collection has items
51         while (iterator.hasNext()) {
52            if (collection2.contains(iterator.next())) {
53               iterator.remove(); // remove current element
54            }
55         }
56      }
57   }
```

```
ArrayList:
MAGENTA RED WHITE BLUE CYAN

ArrayList after calling removeColors:
MAGENTA CYAN
```

Fig. 16.2 | `Collection` interface demonstrated via an `ArrayList` object. (Part 4 of 4.)



Software Engineering Observation

We refer to the `ArrayList`s in this example via `List` variables. This makes our code more flexible and easier to modify—if we later determine that `LinkedList`s would be more appropriate, only the lines where we created the `ArrayList` objects (lines 12 and 20) need to be modified. In general, when you create a collection object, refer to that object with a variable of the corresponding collection interface type. Similarly, implementing method `removeColors` to receive `Collection` references enables the method to be used with any collection that implements the interface `Collection`.

Method	Description
`sort`	Sorts the elements of a `List`.
`binarySearch`	Locates an object in a `List`, using the efficient binary search algorithm
`reverse`	Reverses the elements of a `List`.
`shuffle`	Randomly orders a `List`'s elements.
`fill`	Sets every `List` element to refer to a specified object.
`copy`	Copies references from one `List` into another.

Fig. 16.5 | Some `Collections` methods.

Note: **Java does *not* allow primitive types (like int) as generic type parameters.**

→ List<int> numbers; // □ Not allowed (List<Integer> is correct)

Method	Description
`min`	Returns the smallest element in a `Collection`.
`max`	Returns the largest element in a `Collection`.
`addAll`	Appends all elements in an array to a `Collection`.
`frequency`	Calculates how many collection elements are equal to the specified element.
`disjoint`	Determines whether two collections have no elements in common.

Fig. 16.5 | Some `Collections` methods.

```java
// Fig. 16.6: Sort1.java
// Collections method sort.
import java.util.List;
import java.util.Arrays;
import java.util.Collections;

public class Sort1 {
   public static void main(String[] args) {
      String[] suits = {"Hearts", "Diamonds", "Clubs", "Spades"};

      // Create and display a list containing the suits array elements
      List<String> list = Arrays.asList(suits);
      System.out.printf("Unsorted array elements: %s%n", list);

      Collections.sort(list); // sort ArrayList
      System.out.printf("Sorted array elements: %s%n", list);
   }
}
```

```
Unsorted array elements: [Hearts, Diamonds, Clubs, Spades]
Sorted array elements: [Clubs, Diamonds, Hearts, Spades]
```

Fig. 16.6 | Collections method sort.

```java
// Fig. 16.7: Sort2.java
// Using a Comparator object with method sort.
import java.util.List;
import java.util.Arrays;
import java.util.Collections;

public class Sort2 {
   public static void main(String[] args) {
      String[] suits = {"Hearts", "Diamonds", "Clubs", "Spades"};

      // Create and display a list containing the suits array elements
      List<String> list = Arrays.asList(suits); // create List
      System.out.printf("Unsorted array elements: %s%n", list);

      // sort in descending order using a comparator
      Collections.sort(list, Collections.reverseOrder());
      System.out.printf("Sorted list elements: %s%n", list);
   }
}
```

```
Unsorted array elements: [Hearts, Diamonds, Clubs, Spades]
Sorted list elements: [Spades, Hearts, Diamonds, Clubs]
```

Fig. 16.7 | `Collections` method `sort` with a `Comparator` object.

Generic Classes and Methods:
A Deeper Look

Example of generics and reuse opportunities: **A example with Arrays**

```
1  // Fig. 20.1: OverloadedMethods.java
2  // Printing array elements using overloaded methods.
3
4  public class OverloadedMethods {
5     public static void main(String[] args) {
6        // create arrays of Integer, Double and Character
7        Integer[] integerArray = {1, 2, 3, 4, 5, 6};
8        Double[] doubleArray = {1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7};
9        Character[] characterArray = {'H', 'E', 'L', 'L', 'O'};
10
11       System.out.printf("Array integerArray contains: ");
12       printArray(integerArray); // pass an Integer array
13       System.out.printf("Array doubleArray contains: ");
14       printArray(doubleArray); // pass a Double array
15       System.out.printf("Array characterArray contains: ");
16       printArray(characterArray); // pass a Character array
17    }
```

Fig. 20.1 | Printing array elements using overloaded methods. (Part 1 of 3.)

Example of generics and reuse opportunities: **A example with Arrays (2)**

```
18
19      // method printArray to print Integer array
20      public static void printArray(Integer[] inputArray) {
21         // display array elements
22         for (Integer element : inputArray) {
23            System.out.printf("%s ", element);
24         }
25
26         System.out.println();
27      }
28
29      // method printArray to print Double array
30      public static void printArray(Double[] inputArray) {
31         // display array elements
32         for (Double element : inputArray) {
33            System.out.printf("%s ", element);
34         }
35
36         System.out.println();
37      }
```

Fig. 20.1 | Printing array elements using overloaded methods. (Part 2 of 3.)

Example of generics and reuse opportunities: **A example with Arrays (3)**

- We have an opportunity here to reuse code by creating printArray with arguments of generic types:

```
38
39      // method printArray to print Character array
40      public static void printArray(Character[] inputArray) {
41         // display array elements
42         for (Character element : inputArray) {
43            System.out.printf("%s ", element);
44         }
45
46         System.out.println();
47      }
48   }
```

```
Array integerArray contains: 1 2 3 4 5 6
Array doubleArray contains: 1.1 2.2 3.3 4.4 5.5 6.6 7.7
Array characterArray contains: H E L L O
```

Example of generics and reuse opportunities: **A example with Arrays (4)**

```
1  // Fig. 20.3: GenericMethodTest.java
2  // Printing array elements using generic method printArray.
3
4  public class GenericMethodTest {
5     public static void main(String[] args) {
6        // create arrays of Integer, Double and Character
7        Integer[] integerArray = {1, 2, 3, 4, 5};
8        Double[] doubleArray = {1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7};
9        Character[] characterArray = {'H', 'E', 'L', 'L', 'O'};
10
11       System.out.printf("Array integerArray contains: ");
12       printArray(integerArray); // pass an Integer array
13       System.out.printf("Array doubleArray contains: ");
14       printArray(doubleArray); // pass a Double array
15       System.out.printf("Array characterArray contains: ");
16       printArray(characterArray); // pass a Character array
17    }
```

Fig. 20.3 | Printing array elements using generic method `printArray`. (Part 1 of 2.)

Example of generics and reuse opportunities: **A example with Arrays (5)**

```
18
19      // generic method printArray
20      public static <T> void printArray(T[] inputArray) {
21         // display array elements
22         for (T element : inputArray) {
23            System.out.printf("%s ", element);
24         }
25
26         System.out.println();
27      }
28   }
```

```
Array integerArray contains: 1 2 3 4 5
Array doubleArray contains: 1.1 2.2 3.3 4.4 5.5 6.6 7.7
Array characterArray contains: H E L L O
```

Fig. 20.3 | Printing array elements using generic method `printArray`. (Part 2 of 2.)

Example of generics and reuse opportunities: **A generic Stack example**


```
1  // Fig. 20.7: Stack.java
2  // Stack generic class declaration.
3  import java.util.ArrayList;
4  import java.util.NoSuchElementException;
5
6  public class Stack<E> {
7     private final ArrayList<E> elements; // ArrayList stores stack elements
8
9     // no-argument constructor creates a stack of the default size
10    public Stack() {
11       this(10); // default stack size
12    }
13
14    // constructor creates a stack of the specified number of elements
15    public Stack(int capacity) {
16       int initCapacity = capacity > 0 ? capacity : 10; // validate
17       elements = new ArrayList<E>(initCapacity); // create ArrayList
18    }
```

Fig. 20.7 | Stack generic class declaration. (Part 1 of 2.)

Example of generics and reuse opportunities: **A generic Stack example (2)**

```
19
20      // push element onto stack
21      public void push(E pushValue) {
22         elements.add(pushValue); // place pushValue on Stack
23      }
24
25      // return the top element if not empty; else throw exception
26      public E pop() {
27         if (elements.isEmpty()) { // if stack is empty
28            throw new NoSuchElementException("Stack is empty, cannot pop");
29         }
30
31         // remove and return top element of Stack
32         return elements.remove(elements.size() - 1);
33      }
34   }
```

Fig. 20.7 | Stack generic class declaration. (Part 2 of 2.)

Example of generics and reuse opportunities: **A generic Stack example (3)**

```
1  // Fig. 20.8: StackTest.java
2  // Stack generic class test program.
3  import java.util.NoSuchElementException;
4
5  public class StackTest {
6     public static void main(String[] args) {
7        double[] doubleElements = {1.1, 2.2, 3.3, 4.4, 5.5};
8        int[] integerElements = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
9
10       // Create a Stack<Double> and a Stack<Integer>
11       Stack<Double> doubleStack = new Stack<>(5);
12       Stack<Integer> integerStack = new Stack<>();
13
14       // push elements of doubleElements onto doubleStack
15       testPushDouble(doubleStack, doubleElements);
16       testPopDouble(doubleStack); // pop from doubleStack
17
18       // push elements of integerElements onto integerStack
19       testPushInteger(integerStack, integerElements);
20       testPopInteger(integerStack); // pop from integerStack
21    }
```

Fig. 20.8 | Stack generic class test program. (Part 1 of 6.)

Example of generics and reuse opportunities: **A generic Stack example (4)**

```
22
23      // test push method with double stack
24      private static void testPushDouble(
25         Stack<Double> stack, double[] values) {
26         System.out.printf("%nPushing elements onto doubleStack%n");
27
28         // push elements to Stack
29         for (double value : values) {
30            System.out.printf("%.1f ", value);
31            stack.push(value); // push onto doubleStack
32         }
33      }
34
```

Fig. 20.8 | Stack generic class test program. (Part 2 of 6.)

Example of generics and reuse opportunities: **A generic Stack example (5)**

```
35      // test pop method with double stack
36      private static void testPopDouble(Stack<Double> stack) {
37         // pop elements from stack
38         try {
39            System.out.printf("%nPopping elements from doubleStack%n");
40            double popValue; // store element removed from stack
41
42            // remove all elements from Stack
43            while (true) {
44               popValue = stack.pop(); // pop from doubleStack
45               System.out.printf("%.1f ", popValue);
46            }
47         }
48         catch(NoSuchElementException noSuchElementException) {
49            System.err.println();
50            noSuchElementException.printStackTrace();
51         }
52      }
```

Fig. 20.8 | Stack generic class test program. (Part 3 of 6.)

Example of generics and reuse opportunities: **A generic Stack example (6)**

```
53
54      // test push method with integer stack
55      private static void testPushInteger(
56         Stack<Integer> stack, int[] values) {
57         System.out.printf("%nPushing elements onto integerStack%n");
58
59         // push elements to Stack
60         for (int value : values) {
61            System.out.printf("%d ", value);
62            stack.push(value); // push onto integerStack
63         }
64      }
65
```

Fig. 20.8 | Stack generic class test program. (Part 4 of 6.)

Example of generics and reuse opportunities: **A generic Stack example (7)**

```
66      // test pop method with integer stack
67      private static void testPopInteger(Stack<Integer> stack) {
68         // pop elements from stack
69         try {
70            System.out.printf("%nPopping elements from integerStack%n");
71            int popValue; // store element removed from stack
72
73            // remove all elements from Stack
74            while (true) {
75               popValue = stack.pop(); // pop from intStack
76               System.out.printf("%d ", popValue);
77            }
78         }
79         catch(NoSuchElementException noSuchElementException) {
80            System.err.println();
81            noSuchElementException.printStackTrace();
82         }
83      }
84   }
```

Fig. 20.8 | Stack generic class test program. (Part 5 of 6.)

Example of generics and reuse opportunities: **A generic Stack example (8)**

We have an opportunity for reuse: testPopInteger is very close to testPopDouble (same for Push)

```
Pushing elements onto doubleStack
1.1 2.2 3.3 4.4 5.5
Popping elements from doubleStack
5.5 4.4 3.3 2.2 1.1
java.util.NoSuchElementException: Stack is empty, cannot pop
        at Stack.pop(Stack.java:28)
        at StackTest.testPopDouble(StackTest.java:44)
        at StackTest.main(StackTest.java:16)

Pushing elements onto integerStack
1 2 3 4 5 6 7 8 9 10
Popping elements from integerStack
10 9 8 7 6 5 4 3 2 1
java.util.NoSuchElementException: Stack is empty, cannot pop
        at Stack.pop(Stack.java:28)
        at StackTest.testPopInteger(StackTest.java:75)
        at StackTest.main(StackTest.java:20)
```

Example of generics and reuse opportunities: **A generic Stack example (9)**

```
1  // Fig. 20.9: StackTest2.java
2  // Passing generic Stack objects to generic methods.
3  import java.util.NoSuchElementException;
4
5  public class StackTest2 {
6     public static void main(String[] args) {
7        Double[] doubleElements = {1.1, 2.2, 3.3, 4.4, 5.5};
8        Integer[] integerElements = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
9
10       // Create a Stack<Double> and a Stack<Integer>
11       Stack<Double> doubleStack = new Stack<>(5);
12       Stack<Integer> integerStack = new Stack<>();
13
14       // push elements of doubleElements onto doubleStack
15       testPush("doubleStack", doubleStack, doubleElements);
16       testPop("doubleStack", doubleStack); // pop from doubleStack
17
18       // push elements of integerElements onto integerStack
19       testPush("integerStack", integerStack, integerElements);
20       testPop("integerStack", integerStack); // pop from integerStack
21    }
```

Fig. 20.9 | Passing generic Stack objects to generic methods. (Part 1 of 4.)

Example of generics and reuse opportunities: **A generic Stack example (10)**

```
22
23      // generic method testPush pushes elements onto a Stack
24      public static <E> void testPush(String name , Stack<E> stack,
25         E[] elements) {
26         System.out.printf("%nPushing elements onto %s%n", name);
27
28         // push elements onto Stack
29         for (E element : elements) {
30            System.out.printf("%s ", element);
31            stack.push(element); // push element onto stack
32         }
33      }
34
```

Fig. 20.9 | Passing generic `Stack` objects to generic methods. (Part 2 of 4.)

Example of generics and reuse opportunities: **A generic Stack example (11)**

```
35      // generic method testPop pops elements from a Stack
36      public static <E> void testPop(String name, Stack<E> stack) {
37         // pop elements from stack
38         try {
39            System.out.printf("%nPopping elements from %s%n", name);
40            E popValue; // store element removed from stack
41
42            // remove all elements from Stack
43            while (true) {
44               popValue = stack.pop();
45               System.out.printf("%s ", popValue);
46            }
47         }
48         catch(NoSuchElementException noSuchElementException) {
49            System.out.println();
50            noSuchElementException.printStackTrace();
51         }
52      }
53   }
```

Fig. 20.9 | Passing generic Stack objects to generic methods. (Part 3 of 4.)

Example of generics and reuse opportunities: **A generic Stack example (12)**

```
Pushing elements onto doubleStack
1.1 2.2 3.3 4.4 5.5
Popping elements from doubleStack
5.5 4.4 3.3 2.2 1.1
java.util.NoSuchElementException: Stack is empty, cannot pop
        at Stack.pop(Stack.java:28)
        at StackTest2.testPop(StackTest2.java:44)
        at StackTest2.main(StackTest2.java:16)

Pushing elements onto integerStack
1 2 3 4 5 6 7 8 9 10
Popping elements from integerStack
10 9 8 7 6 5 4 3 2 1
java.util.NoSuchElementException: Stack is empty, cannot pop
        at Stack.pop(Stack.java:28)
        at StackTest2.testPop(StackTest2.java:44)
        at StackTest2.main(StackTest2.java:20)
```

Fig. 20.9 | Passing generic Stack objects to generic methods. (Part 4 of 4.)

Exercise:
Implement a generic ADT (abstract data type)

- A **Bag** (also called a *multiset*) is an abstract data type that stores elements **without order**, and **allows duplicates**. For example, a bag could contain {apple, apple, orange}.
- Your task is to design and implement this **generic Bag ADT** in Java.
  - Create an abstract interface Bag
  - Create a concrete implementation using an array

Solution

```
public interface IBag<T> extends Iterable<T> {

    /** Adds an element to the bag. Does not return anything if it succeeds. @param element  */

    public void add(T element) throws IllegalArgumentException, IllegalUseOfBagException;


    /** Removes the exact element from the Bag if it exists.  Uses ( == ) comparison semantics to
  find the element.  */

    public void remove(T element) throws IllegalArgumentException;

    /** Removes all the occurences of element from the Bag if it exists. Uses ( .equals )
  comparison semantics to find the equal elements.  */

    public void removeAll(T element) throws IllegalArgumentException;

    public boolean isEmpty();

    public int size();

    public boolean exists(T element) throws IllegalArgumentException;

}
```

Solution (2)

```
public class ListBag<T> implements IBag<T>{
    private ArrayList<T> elements;
    public ListBag() { elements = new ArrayList<T>(); }
    public ListBag(T... content) {
              elements = new ArrayList<T>(Arrays.asList(content));
    }
    @Override
    public void add(T element) throws IllegalUseOfBagException {
        if(exists(element))
           throw new IllegalUseOfBagException("Cannot add an element more than once");
        if(element != null) { elements.add(element); }
        throw new IllegalArgumentException("Cannot add a null element");
    }
```

Solution (2)

```
@Override
public void remove(T element) {
    if(element != null) {
        Iterator<T> it = elements.iterator();
        while(it.hasNext()) {
            T current = it.next();
            if(current == element) {
                it.remove();
                break;
            }
        }
    }
    throw new IllegalArgumentException("Cannot remove a null element");
}
```

```
@Override
public void removeAll(T element) {
    if(element != null) {
        Iterator<T> it = elements.iterator();
        while(it.hasNext()) {
            T current = it.next();
            if(current.equals(element)) {
                it.remove();
            }
        }
    }
    throw new IllegalArgumentException("Cannot remove a null element");
}
```

Solution (3)

```
@Override
public boolean isEmpty() { return elements.isEmpty(); }
@Override
public int size() { return elements.size(); }
public boolean exists(T element) {
    if(element != null) {
        Iterator<T> it = elements.iterator();
        while(it.hasNext()) {
            T current = it.next();
            if(current == element) return true;
        }
        return false;
    }
    throw new IllegalArgumentException("Argument element cannot be null");
}
```

Solution (4)

```
@Override
public Iterator<T> iterator() {
    return new Iterator<T>() {
        private final int[] shuffledIndices;
        private int currentPosition = 0;
        private final int expectedSize = elements.size();
        // Instance initializer: shuffles indices
        {
            int size = elements.size();
            shuffledIndices = new int[size];
            // Initialize indices: 0, 1, 2, ..., size-1
            for (int i = 0; i < size; i++) {
                shuffledIndices[i] = i;
            }
            Random random = new Random();
            for (int i = size - 1; i > 0; i--) {
                int j = random.nextInt(i + 1);
                int temp = shuffledIndices[i];
                shuffledIndices[i] = shuffledIndices[j];
                shuffledIndices[j] = temp;
            }
        }
        @Override
        public boolean hasNext() { return currentPosition < shuffledIndices.length; }
        @Override
        public T next() {
            if (!hasNext()) throw new NoSuchElementException("No more elements in bag");
            return elements.get(shuffledIndices[currentPosition++]);
        }
    };
}
```

Example of generics and reuse opportunities: **Using Wildcards Generics**

```
1  // Fig. 20.11: WildcardTest.java
2  // Wildcard test program.
3  import java.util.ArrayList;
4  import java.util.List;
5
6  public class WildcardTest {
7     public static void main(String[] args) {
8        // create, initialize and output List of Integers, then
9        // display total of the elements
10       Integer[] integers = {1, 2, 3, 4, 5};
11       List<Integer> integerList = new ArrayList<>();
12
13       // insert elements in integerList
14       for (Integer element : integers) {
15          integerList.add(element);
16       }
17
```

Fig. 20.11 | Wildcard test program. (Part 1 of 4.)

Example of generics and reuse opportunities: **Using Wildcards Generics(2)**

```
18          System.out.printf("integerList contains: %s%n", integerList);
19          System.out.printf("Total of the elements in integerList: %.0f%n%n",
20             sum(integerList));
21
22          // create, initialize and output List of Doubles, then
23          // display total of the elements
24          Double[] doubles = {1.1, 3.3, 5.5};
25          List<Double> doubleList = new ArrayList<>();
26
27          // insert elements in doubleList
28          for (Double element : doubles) {
29             doubleList.add(element);
30          }
31
32          System.out.printf("doubleList contains: %s%n", doubleList);
33          System.out.printf("Total of the elements in doubleList: %.1f%n%n",
34             sum(doubleList));
35
```

Fig. 20.11 | Wildcard test program. (Part 2 of 4.)

Example of generics and reuse opportunities: **Using Wildcards Generics(3)**

```
36          // create, initialize and output List of Numbers containing
37          // both Integers and Doubles, then display total of the elements
38          Number[] numbers = {1, 2.4, 3, 4.1}; // Integers and Doubles
39          List<Number> numberList = new ArrayList<>();
40
41          // insert elements in numberList
42          for (Number element : numbers) {
43             numberList.add(element);
44          }
45
46          System.out.printf("numberList contains: %s%n", numberList);
47          System.out.printf("Total of the elements in numberList: %.1f%n",
48             sum(numberList));
49       }
50
```

Fig. 20.11 | Wildcard test program. (Part 3 of 4.)

Example of generics and reuse opportunities: **Using Wildcards Generics(4)**

```
51      // total the elements; using a wildcard in the List parameter
52      public static double sum(List<? extends Number> list) {
53         double total = 0; // initialize total
54
55         // calculate sum
56         for (Number element : list) {
57            total += element.doubleValue();
58         }
59
60         return total;
61      }
62   }
```

```
integerList contains: [1, 2, 3, 4, 5]
Total of the elements in integerList: 15

doubleList contains: [1.1, 3.3, 5.5]
Total of the elements in doubleList: 9.9

numberList contains: [1, 2.4, 3, 4.1]
Total of the elements in numberList: 10.5
```

Fig. 20.11 | Wildcard test program. (Part 4 of 4.)

Exercises

Exercise 1 – Generic Class

1. Create a generic class `Box<T>` where `T` is the type parameter.
2. Add the following methods to the class:
   - `void set(T item)`: Stores an item in the box.
   - `T get()`: Retrieves the stored item.
3. Show how the Box class is used with different data types (e.g., Integer, String, Double).

Exercise: Generic Method

- Write a method to swap two items in the Pair class

```
public Pair<U, T> swap() {
   return new Pair<>(second, first);
}
```

```java
public class Pair2<T, S> {

    private T first;
    private S second;

    public Pair2() {
    }

    public Pair2(T first, S second) {
        this.first = first;
        this.second = second;
    }

    public T getFirst() {
        return first;
    }

    public S getSecond() {
        return second;
    }

    public void setFirst(T newValue) {
        this.first = newValue;
    }

    public void setSecond(S newValue) {
        this.second = newValue;
    }

    public Pair2<S, T> swap() {
        return new Pair2<S, T>(second, first);
    }
}
```

```java
public class MainApp {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        Pair2<String, Integer> p = new Pair2<>("Age", 25);
        Pair2<Integer, String> result = p.swap();
        System.out.println("swap results: ");
        System.out.println("First: " + p.getFirst());
        System.out.println("Second: " + p.getSecond());
    }

}
```

Exercise Generic Interface

```
//Syntax: interface InterfaceName<T>
public interface Pair<K, V> {
    K getKey();
    V getValue();
}

//Implementation
class KeyValue<K, V> implements Pair<K, V> {
    private K key;
    private V value;

    public KeyValue(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey() {
        return key;
    }

    public V getValue() {
        return value;
    }
}


//Usage
Pair<Integer, String> pair = new
KeyValue<>(1, "One");
System.out.println(pair.getKey() + " = " +
pair.getValue());
```

Example: Passing Lambda as argument

- Lambdas can be passed to methods expecting functional interfaces, allowing you to define custom behaviors on the fly.

```
@FunctionalInterface
interface Operation {
    int execute(int a, int b);
}

public class LambdaExample {
    public static void main(String[] args) {
    // Passing a lambda expression as an argument
    int result = performOperation(5, 3, (a, b) -> a + b);
    System.out.println("Result: " + result); // Output: Result: 8
    }

static int performOperation(int a, int b, Operation operation) {
    return operation.execute(a, b);
    }
}
```

Example: Returning Lambdas from Methods

- Lambdas can also be returned by methods, making it easy to generate custom behaviors.

```
import java.util.function.Function;

public class LambdaReturnExample {
   //Method returning a lambda that squares a number
   public static Function<Integer, Integer> getSquareLambda() {
     return (Integer x) -> x * x;
     //Returning a lambda
     }

    public static void main(String[] args) {
       Function<Integer, Integer> square = getSquareLambda();
       System.out.println("Square of 5: " + square.apply(5));//Output: 25
    }
}
```

Learning Outcomes

- By now, you should be able to
- Explain the concepts of generics and lambda expressions
- Evaluate generics and lambda expressions in terms of best practices
- Appreciate how generics and lambda expressions help improve the software construction quality
- Analyze and improve existing or new software projects using generics and lambda expressions